

Chatbox on ARC: Technical Plan and Software Design for In-Chat Reasoning Inspection

Connecting the Chatbox Community Edition to Virginia Tech ARC Inference Services and Implementing the Fifteen-Technique Catalog

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

This document plans and designs a research fork of the Chatbox Community Edition that connects to Virginia Tech Advanced Research Computing (ARC) inference services and implements the fifteen in-chat reasoning inspection techniques cataloged in the companion paper "Fifteen Visualization and Interaction Techniques for Inspecting and Intervening in LLM Reasoning Within the Chat Interface." The design covers four subsystems: an ARC provider layer that reaches GLM-5.2 today and Kimi K3 when ARC deploys it, across three connection profiles (the centralized llm-api endpoint, dedicated Open OnDemand sessions, and self-managed vLLM on Slurm); a Reasoning Augmentation Service that runs the expensive operations (resampling, entailment clustering, knockout probes, second opinions) on ARC hardware next to the model rather than inside Electron; a renderer-side technique layer that adds the visual encodings and interactions to the chat frame; and a session state store for the techniques that persist across turns. The document then specifies the configuration management approach for a public repository under github.com/nathanconklin, an agentic development workflow built on the fork's existing Claude Code skill directory, a feature-branch model with one branch per technique, and a five-phase implementation sequence ordered by the priority argument in the companion paper. Risks with concrete mitigations close the document, the largest being that the centralized ARC API does not expose the token-level access that Techniques 1 through 3 require, which makes the vLLM profile a hard dependency for the strongest publication candidates.

1. Purpose and Scope

The companion paper catalogs fifteen interaction techniques and closes with the observation that the methods exist and the interface does not. This document is the bridge from catalog to code. It selects Chatbox Community Edition as the host application, specifies how the fork reaches ARC-hosted models, assigns each technique to a subsystem and a feature branch, and defines the workflow that an agentic development process will execute.

Three constraints shape every decision below.

First, the techniques divide by the layer of model access they consume. Techniques 6, 7, 9, and 10 need structured output from an ordinary chat completion. Techniques 1, 2, and 12 need per-token log probabilities. Techniques 3, 4, 5, 8, 11, 13, 14, and 15 need additional inference: resampling, ablated re-queries, or a second reasoner. The connection profile determines which layers are reachable, so the provider design and the technique schedule are coupled.

Second, Chatbox is an Electron application written in TypeScript. Entailment clustering, batch resampling, and context ablation do not belong in a desktop renderer process. The design pushes that work into a Python sidecar on ARC and keeps the Electron changes confined to rendering, interaction, and session state.

Third, the project is a research vehicle. Each technique in the paper names a dependent variable; the implementation must log the events that make those variables measurable, or the build produces a demo instead of an instrument.

Out of scope: mobile builds, the Chatbox team-sharing subsystem, upstream contribution of the research features (the fork may contribute bug fixes upstream, but the technique layer stays in the fork), and the reasoning-structure extraction problem, which the companion paper defers to separate treatment. Where a technique needs a reasoning graph, this design uses structured-output prompting as a provisional extractor and isolates it behind an interface so a better extractor can replace it.

2. System Context

2.1 Chatbox Community Edition Baseline

Chatbox (github.com/chatboxai/chatbox) is a GPL-3.0 desktop client for LLM providers, built on Electron with a React renderer, packaged with pnpm, linted with Biome, and tested with Vitest and Storybook. The architecture splits into `src/main` (Electron main process), `src/renderer` (React UI), `src/preload`, and `src/shared` (code used by both processes). Three parts of the codebase carry this project:

Area	Path	Role in this project
Provider definitions	<code>src/shared/providers/definitions/</code>	One file per provider (<code>claude.ts</code> , <code>gemini.ts</code> , <code>chatglm.ts</code> , <code>azure.ts</code>). The ARC provider lands here.
Model implementations	<code>src/shared/models/</code>	<code>openai-compatible.ts</code> implements the OpenAI wire protocol the ARC endpoints speak. The ARC model class extends it.
Call orchestration	<code>src/renderer/packages/model-calls/</code>	Preprocessing, tool integration, and streaming. Reasoning-payload handling hooks in here.

Two properties of the baseline lower the project's risk. The repository already ships a `.claude/skills` directory, so the agentic workflow slots into an existing convention rather than fighting one. And the evaluation that selected Chatbox confirmed the OpenAI-compatible path is the primary integration surface, which matches all three ARC connection profiles.

The fork inherits the GPL-3.0 license. The project repository stays public with source available, which satisfies the license and matches the research goal.

2.2 ARC Inference Services

ARC exposes three ways to reach hosted models, documented in the `LLM_on_ARC_SYSTEMS` workshop materials and ARC's AI documentation (docs.arc.vt.edu/zz_ai.html). All three run on premises; no prompt or document leaves the university, which is why the services carry approval for high-risk data classes.

Profile	Endpoint	Backend	Auth	Limits
Centralized API	<code>https://llm-api.arc.vt.edu/api/v1</code>	Shared cluster	API key from <code>llm.arc.vt.edu</code>	60 req/min, 1,000/hr, 3,000 per 3-hr window
OOD dedicated LLM	Per-session URL from <code>ood.arc.vt.edu</code>	<code>llama.cpp</code> on one GPU	Per-session key	None; 5-day max, 1-hr idle timeout, consumes SUs
vLLM on Slurm	Self-managed node/port	vLLM, tensor parallel	Self-managed	None; allocation-bounded

The centralized API currently serves `gpt-oss-120b`, `Kimi-K2.6`, and `GLM-5.2`. `GLM-5.2` is the initial research target; `Kimi K3` replaces or joins it when ARC deploys the model. Pre-downloaded weights live at `/common/data/models/` on the clusters, which is where a vLLM launch will look first.

One networking detail from the workshop materials matters for automation: a Jupyter or batch session on TinkerCliffs cannot reach an OOD LLM on a Falcon GPU node directly; the workshop bridges the gap with an SSH tunnel through `falcon1.arc.vt.edu`. The desktop client faces the same topology from off-cluster, so the ARC provider must treat tunnel setup as a first-class configuration concern rather than an afterthought. All profiles additionally require the VT network or VPN.

2.3 Access Layers by Profile

The profiles differ in which layers of the companion paper's access model they expose, and that difference drives the technique schedule.

Capability	Centralized API	OOD llama.cpp	vLLM
Chat completions, streaming	Yes	Yes	Yes
Structured output (JSON schema)	Yes	Yes	Yes (guided decoding)
Per-token logprobs with top-k alternates	Not exposed through the Open WebUI API layer	Yes (logprobs, top_logprobs)	Yes, plus hidden-state instrumentation
High-volume resampling	Rate-limited	Single-GPU throughput	Batch-optimized
Model choice	Fixed menu	OOD dropdown	Anything in <code>/common/data/models/</code>

The consequence: Techniques 6, 7, 9, 10, and 13 run against any profile, including the centralized API a user already has. Techniques 1, 2, and 12 require the OOD or vLLM profile. Techniques 3, 4, 5, 8, 11, 14, and 15 run anywhere in principle but are only practical where resampling does not eat a shared rate budget; five samples per decision node at 60 requests per minute leaves no headroom for the conversation itself. The vLLM profile is therefore the research configuration, and the companion paper's claim that the self-hosted deployment is a research asset becomes a concrete dependency here.

3. Software Design

3.1 Architecture Overview

The system adds four subsystems to the Chatbox baseline. Ordered from the model outward:

1. **ARC Provider Layer** (TypeScript, `src/shared/providers/definitions/arc.ts` plus a model class extending `openai-compatible.ts`). Encapsulates the three connection profiles, credential storage, model discovery, and profile-specific capability flags.
2. **Reasoning Augmentation Service, RAS** (Python, FastAPI, deployed on ARC next to vLLM; separate repository directory `services/ras/`). An OpenAI-compatible passthrough proxy that adds `/reasoning/*` endpoints for the expensive operations: resample-and-cluster, knockout probe, divergence analysis, second opinion, and structure extraction. Chatbox talks to RAS; RAS talks to vLLM.
3. **Technique Layer** (TypeScript/React, `src/renderer/components/reasoning/`). One component family per technique, each behind a feature flag, sharing a common annotation model that maps reasoning data onto response spans.
4. **Session Reasoning Store** (TypeScript, extending Chatbox's session model in `src/shared/types` and the renderer session stores). Persists assumptions, open questions, probe verdicts, and per-turn reasoning snapshots so Techniques 6 through 9 survive across turns and across application restarts.

The split follows one rule: anything that costs GPU time or needs Python-ecosystem tooling (sentence embeddings, NLI models for entailment) lives in RAS; anything that renders or persists lives in Chatbox.

3.2 ARC Provider Layer

The provider definition registers "Virginia Tech ARC" as a selectable provider with three profile presets. Each profile carries a capability descriptor the technique layer reads at runtime:

```
interface ArcProfileCapabilities {
  logprobs: boolean      // per-token distributions available
  topLogprobs: number    // max alternates per position
  unmeteredSampling: boolean // resampling will not exhaust a shared budget
  rasEndpoint?: string   // Reasoning Augmentation Service base URL
  guidedDecoding: boolean // JSON-schema-constrained output
}
```

Techniques gate on capabilities, not on profile names. A user on the centralized API sees Techniques 6, 7, 9, 10, and 13 active and the rest greyed with an explanation of what access they need; switching to the vLLM profile lights up the rest. This turns the paper's layered-access argument into a visible property of the interface, which is itself worth a screenshot in the eventual write-up.

Credential handling: API keys enter through the existing Chatbox settings UI and persist in the platform keychain via Electron safeStorage, never in the repository or in exported session files. The OOD profile accepts the per-session URL and key from the OOD session card; the provider surfaces the 1-hour idle timeout as a countdown in the connection status so a dead session reads as expired rather than broken. The vLLM profile accepts a host and port and an optional SSH tunnel spec (user@falcon1.arc.vt.edu), and the Electron main process owns the tunnel lifecycle.

Model discovery uses the standard /models listing on all three profiles. GLM-5.2 and Kimi K3 need no special casing beyond reasoning-content handling: both emit chain-of-thought in a reasoning_content field under the OpenAI-compatible wire protocol, and the model class must capture it as a distinct message part rather than concatenating it into the answer, since the technique layer annotates the two differently.

3.3 Reasoning Augmentation Service

RAS exists because the knockout probe, resampling, and second opinions are batch inference jobs, and a desktop client should submit them, not run them. It is a FastAPI application with two faces:

Passthrough face. /v1/chat/completions forwards to the configured vLLM server unchanged, adding request logging keyed by a conversation ID the client supplies. Chatbox points its ARC vLLM profile at RAS instead of at vLLM directly, so every response RAS later ablates or resamples is one it already saw. This removes a whole class of replay bugs.

Reasoning face. Versioned endpoints, one per operation:

Endpoint	Backing method	Serves technique
POST /reasoning/ resample	N re-generations at temperature, entailment clustering per Farquhar et al. (bidirectional NLI with a small local model), cluster counts and exemplar traces returned	4, 5
POST /reasoning/ knockout	Re-ask with one cited context item removed; sentence-level diff of the two answers; verdict (changed / held / partial)	3
POST /reasoning/ second- opinion	Same claim to a second model or second framing; span-level agreement report	15

Endpoint	Backing method	Serves technique
POST /reasoning/ extract	Structured-output pass that emits the provisional reasoning graph: claims, assumptions, dependencies, open questions, per-node certainty	6, 7, 8, 10, and the consequence-weight computation
GET /reasoning/ logprobs/{responseId}	Cached per-token logprobs and top-k alternates from the passthrough capture	1, 2, 12

Design positions worth stating. Clustering starts with the entailment method because it carries published discrimination results; the embedding alternative stays behind the same interface for the over-merge pilot the paper calls for. The extractor is provisional and everything downstream treats its output as noisy; RAS attaches an extraction-confidence field per node so the ledger can render low-confidence assumptions differently rather than assert them. All reasoning-face operations are on-demand, matching the paper's Section 3.2 cost position, with one exception: the passthrough face may launch a speculative resample in the background while the user reads the first response, controlled by a server-side config flag and off by default until latency measurement on the target hardware sets the thresholds.

RAS ships with a Slurm launch script (`services/ras/slurm/launch.sh`) that starts vLLM with `--logprobs` enabled against a model directory under `/common/data/models/`, starts RAS on the same node, and prints the tunnel command. One script, one node, both processes; teardown is `scancel`.

3.4 Technique Layer

The renderer work shares three pieces of infrastructure so fifteen features do not become fifteen bespoke rendering paths.

Annotation model. A single `ReasoningAnnotation` type maps every signal (token entropy, stated certainty, probe verdict, cluster split, assumption reference) onto character ranges of the response text or onto node IDs of the extracted graph. Techniques are renderers over this model. This is where the paper's encoding-budget discussion becomes code: the model carries the quantity, and a central encoding registry assigns the channel (density for entropy, color for certainty, badges for verdicts, position for the gutter), so a channel conflict is a registry error rather than a discovery during review.

Feature flags and settings. Each technique registers under `settings.reasoning.techniques[n]` with an enabled flag, its capability requirements, and its per-technique options (sample count for 4 and 5, certainty floor default for 12). A settings panel groups them by the paper's four problem groups.

Instrumentation. A local event log (JSONL in the app data directory, no network export by default) records the events behind each technique's dependent variable: probe invocations and verdicts, ledger accept/reject/edit actions, queue opens and closures, threshold slider positions, frame-card edits, mid-stream corrections. Log schema versioning starts at v1 in Phase 1 because retrofitting instrumentation after a study design exists is the wrong order.

Component notes for the techniques with renderer subtlety:

- **Hesitation Ribbon (1):** an SVG underlay per response line, density-modulated from the logprob cache; hover opens the top-k popover. Rendering cost is bounded by virtualizing to visible lines.
- **Cross-Layer Agreement Gutter (2):** a fixed-width margin strip; each mark subscribes to one annotation channel. The legibility pilot the paper requires gets a Storybook story with a size matrix before any user ever sees it.
- **Assumption Ledger (6) and Open Questions Queue (7):** side panels bound to the Session Reasoning Store; the drift check runs on each extract result against the stored set, and a conflict banner names both turns.
- **Frame Card (10):** rendered from a pre-generation extract pass as a collapsed strip above the streaming response, expanded on hover; the collapsed-strip rendering is the load-bearing decision and gets its own flag so the study can compare strip against full card.

- **Mid-Stream Intervention Window (11):** depends on streaming the extract pass concurrently with generation; the correction injects a user message and resumes rather than restarting, using the passthrough face's conversation state.
- **Certainty Threshold Rendering (12):** a pure renderer over stated-certainty annotations; collapsed spans keep a visible expand affordance per the paper's honesty risk.

3.5 Session Reasoning Store

Chatbox persists sessions locally already; this store extends the session schema with a **reasoning** block: the assumption set with states and origin turns, the open-questions list, probe verdicts keyed by citation, and a per-turn snapshot of the extracted graph for Technique 8's diff. Schema changes ship with a migration under the existing `src/shared/migration/` convention so pre-existing user sessions load cleanly. Export and import of a session include the reasoning block, which makes study sessions portable artifacts.

4. Technique-to-Branch Mapping

Each technique gets one feature branch, one settings flag, and one instrumentation event set. Shared infrastructure lands first on its own branches.

Branch	Delivers	Depends on
feature/arc-provider	Provider, profiles, capabilities, tunnel handling	none
feature/ras-core	RAS passthrough, logprob capture, Slurm launch	ARC allocation
feature/reasoning-store	Session schema, migration, annotation model, flags, instrumentation	none
feature/ui-frame-card	Technique 10	store, extract endpoint
feature/ui-assumption-ledger	Technique 6	store, extract endpoint
feature/ui-open-questions	Technique 7	ledger
feature/ui-conversation-spine	Technique 9	store
feature/ui-action-ribbon	Technique 13 (timeline first, rewrite interaction second)	store
feature/ui-hesitation-ribbon	Technique 1	ras-core logprobs
feature/ui-threshold-rendering	Technique 12	annotation model
feature/ui-agreement-gutter	Technique 2	1's channel plus verifier inputs
feature/ras-resample + feature/ui-path-distribution	Techniques 4, 5	ras-core
feature/ras-knockout + feature/ui-knockout	Technique 3	ras-core
feature/ui-reasoning-diff	Technique 8	per-turn snapshots; shares matcher with 15
feature/ui-midstream	Technique 11	streaming extract
feature/ui-provenance-descent	Technique 14	action ribbon, citations
feature/ras-second-opinion + feature/ui-overlay	Technique 15	matcher from 8

5. Development Workflow

5.1 Agentic Process

Development runs through Claude Code against the fork, and the process is spec-driven, which is fitting given that Technique 10 argues for exactly this pattern. The repository conventions:

- `‘CLAUDE.md’` at the root: build commands (`pnpm run dev`, `pnpm run lint`, `pnpm run test`), architecture map, the annotation-model contract, and the rule that no technique component reads provider internals directly; everything goes through capabilities and annotations.
- `‘.claude/skills/’`: the fork already ships this directory. Project skills added: `arc-connection` (profile setup, tunnel commands, endpoint sanity checks), `technique-spec` (the template below), and `instrumentation` (event schema and how to add an event without breaking the log version).
- **Per-technique specs** in `docs/specs/NN-technique-name.md`, written before the branch opens: the visual encoding, the interaction, the capability requirements, the dependent-variable events, acceptance criteria, and the Storybook states to produce. The agent implements against the spec; review compares output to spec, which shortens review and leaves a paper trail the eventual publication can cite as method.
- **Agent task granularity**: one branch, one spec, one PR. The agent runs lint and the Vitest suite before proposing the PR; Storybook stories are part of the acceptance criteria for every renderer component, since the paper's design risks (gutter legibility, ribbon density) are testable as static states before any study.

5.2 Branching, Review, and CI

- `main` is protected: PRs only, linear history, required status checks.
- `develop` integrates finished features between milestone tags; feature branches cut from and merge back to `develop`.
- Branch names follow the table in Section 4: `feature/arc-*`, `feature/ras-*`, `feature/ui-*`, plus `fix/*` and `docs/*`.
- Conventional Commits throughout, which keeps the changelog generatable.
- GitHub Actions on every PR: Biome lint, Vitest, TypeScript build, Storybook build; a second workflow runs RAS's pytest suite and ruff when `services/ras/**` changes. A weekly job builds the Electron package on Linux and Windows to catch platform drift early.
- Upstream sync: `upstream` remote pointed at `chatboxai/chatbox`; a monthly `chore/upstream-sync` branch merges upstream `main` into `develop`. The technique layer's isolation (new directories, capability gating, minimal edits to shared files) is what keeps these merges small, and that isolation is a standing review criterion.

5.3 Repository and Configuration Management

The project lives at github.com/nathanconklin/chatbox-reasoning, created as a GitHub fork of `chatboxai/chatbox` so the upstream relationship is native. Layout:

```
chatbox-reasoning/
├── src/                                # Chatbox app (upstream layout preserved)
│   └── renderer/components/reasoning/  # technique layer
├── services/ras/                       # Reasoning Augmentation Service (FastAPI)
│   ├── ras/                           # application package
│   ├── tests/
│   └── slurm/launch.sh                 # vLLM + RAS single-node launcher
├── docs/
│   ├── specs/                         # per-technique specs
│   └── adr/                           # architecture decision records
```



```
└─ arc-setup.md          # profile setup, VPN, tunnels, OOD walkthrough
└─ .claude/skills/       # upstream skills + project skills
└─ CLAUDE.md
└─ .env.example          # names only, no values
```

Secrets policy: ARC API keys and session keys live in the OS keychain (app) or environment variables (RAS); `.env` is gitignored; a gitleaks pre-commit hook and CI step enforce the policy mechanically. Releases are tagged per milestone (`v0.1.0-arc-connect` onward) with packaged builds attached, and each tag freezes the spec set it implements, which gives every study a citable software version.

6. Implementation Phases

The sequence follows the companion paper's priority argument: connectivity first, then the cheap high-effect techniques (6, 10), then the token-level and sampling techniques that carry the strongest publication claims (2, 3, plus 8 and 12), then the rest.

Phase	Contents	Exit criteria
0. Connect	<code>arc-provider</code> , <code>reasoning-store</code> , <code>docs/arc-setup</code>	GLM-5.2 chat over all three profiles from a packaged build; capability flags verified against each
1. Frame and state	<code>ras-core</code> (passthrough + extract), Techniques 10, 6, 7, 9	Drift conflict correctly names both turns in a scripted session; instrumentation log v1 complete
2. Token level	Techniques 1, 12; groundwork for 2	Ribbon renders live vLLM logprobs; divergence case (high stated certainty over high entropy) flagged on a constructed example; threshold slider produces the length curve
3. Sampling and probes	RAS resample and knockout; Techniques 4, 5, 3; then 2	Entailment clustering reproduces published behavior on a pilot set; knockout verdict badges persist per session; gutter passes the size-matrix legibility check
4. Traversal and comparison	Techniques 13, 8, 14, 11, 15	Query-rewrite rerun works end to end; 8 and 15 share one matcher behind one interface
5. Study readiness	Kimi K3 onboarding when available; latency measurement and thresholds; export tooling for logs and sessions	A pilot session yields every dependent variable named in the paper from the logs alone

Phase 0 and 1 run on the centralized API alone, which means development starts before any allocation-consuming session exists. GPU-dependent phases (2 onward) budget SUs explicitly, and every OOD or Slurm session teardown is part of the working procedure, not a cleanup afterthought; the idle timeout makes forgotten sessions cheap but not free.

7. Risks and Mitigations

Risk	Impact	Mitigation
Centralized API never exposes logprobs	Techniques 1, 2, 12 blocked on that profile	Designed in: capability gating; vLLM profile is the research configuration. Verify the assumption empirically in Phase 0 and record the result in an ADR.

Risk	Impact	Mitigation
Rate limits make sampling techniques unusable on the shared API	Techniques 3, 4, 5, 15 impractical for casual users	Same gating; the paper can state the access boundary as a finding rather than hide it.
Kimi K3 timing	Model-comparison plans slip	GLM-5.2 carries all phases; K3 onboarding is a Phase 5 line item with no upstream dependents.
Structure extraction is noisy	Ledger flags false conflicts; diff reports phantom changes	Extraction-confidence field, conservative conflict thresholds, provisional-extractor interface; the paper already scopes extraction out, and the ADR trail documents the provisional choice.
OOD idle timeout and SU burn	Dead sessions mid-development; allocation drain	Session countdown in the provider UI; teardown in the working procedure; centralized API for all non-GPU development.
Tunnel and VPN friction	Setup cost deters use beyond the author	<code>docs/arc-setup.md</code> with copy-paste commands; tunnel lifecycle owned by the app; RAS launch script prints its own tunnel line.
Upstream drift	Painful monthly merges	Technique-layer isolation as a review criterion; monthly sync cadence keeps deltas small.
Electron rendering cost of per-token annotation	Sluggish chat frame defeats the in-chat premise	Virtualized ribbon rendering; annotation model benchmarks in Vitest; Technique 12 and the gutter render per-span, not per-token.
GPL-3.0 obligations	License noncompliance	Public fork, license preserved, RAS released under the same terms; no proprietary components.

8. Summary

The design turns the companion paper's catalog into a buildable system with four subsystems and a clean division of labor: Chatbox renders and persists, RAS computes, ARC hosts, and capability flags tell the user which techniques their connection can support. The workflow is spec-driven and agentic, the repository is public and configuration-managed under github.com/nathanconklin, and the phase order front-loads the connectivity everyone needs and the two techniques the paper judges cheapest and most likely to show an effect. The largest technical bet is stated plainly: the strongest publication candidates require the self-hosted vLLM path, so the project treats that deployment as core infrastructure from Phase 2 forward and measures its latency budget before promising interactions that depend on it.